

P3DX Data Collection Stack

How ROS, hardware drivers, and Docker work together

to collect robot datasets

What is ROS?

ROS (Robot Operating System) is not an OS.

It is a **message-passing middleware**. A system that lets independent programs running on the same machine (or network) talk to each other through named channels.

Think of it like a pub/sub message bus where each program only needs to know the *name* and *type* of the channel it cares about — not who else is on the other end.

Without ROS	With ROS
Every program must know about every other program	Programs are decoupled — publish/subscribe to named topics
Custom serial/socket protocols per device	Standardised message types shared across all drivers
Restart everything to swap a component	Swap one node without touching the rest

Core Concepts

Node

A single running process with a specific job.
Each driver, each script = one node.

Examples in this stack:

- `rosaria` — robot motor driver
- `realsense2_camera` — camera driver
- `teleop_joystick` — joystick reader
- `p3dx_data_recorder` — frame saver

Topic

A named, typed channel.
Nodes publish to it or subscribe from it.

Examples in this stack:

- `/camera/color/image_raw`
- `/RosAria/cmd_vel`
- `/RosAria/pose`

Message

A typed struct carried on a topic.
Defined by ROS packages, reused everywhere.

roscore — The Backbone

`roscore` is the **central name registry**. Every node connects to it on startup to:

- Announce what topics it publishes
- Find what topics it wants to subscribe to

```
roscore running at http://192.168.1.10:11311
```

```
Node A says: "I publish /camera/color/image_raw"
```

```
Node B says: "I want to subscribe to /camera/color/image_raw"
```

```
roscore connects them → messages flow directly A → B
```

roscore must be the first thing started.

Nothing in ROS works without it. If roscore dies, all communication stops.

ROS Message Types

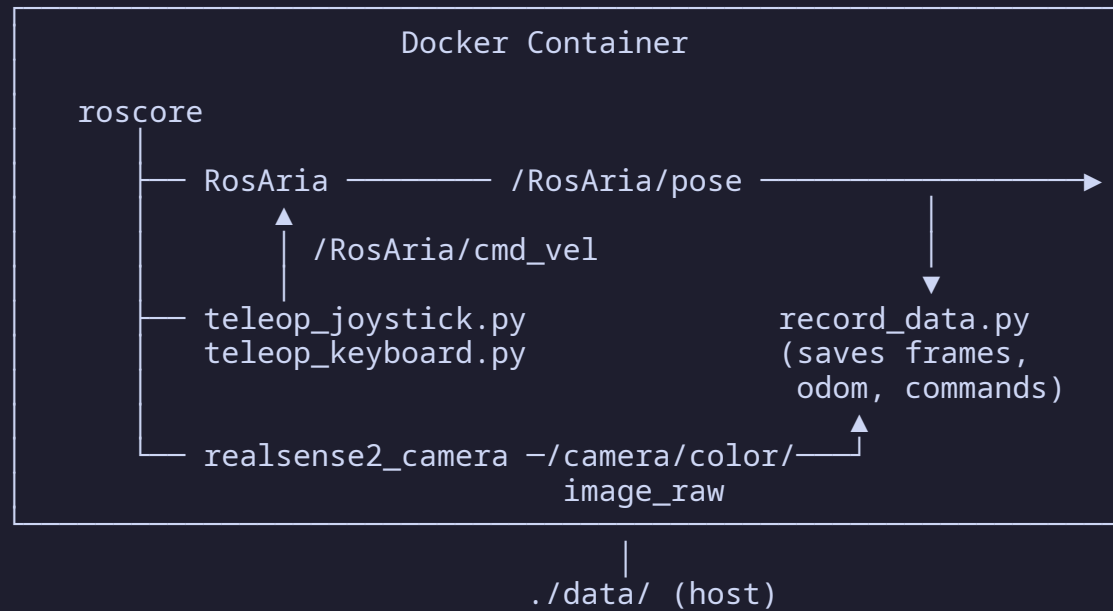
ROS defines standard message types shared across all packages.

Message Type	Fields	Used for
<code>sensor_msgs/Image</code>	<code>data[]</code> , <code>header.stamp</code> , <code>encoding</code>	Camera frames
<code>nav_msgs/Odometry</code>	<code>pose.pose.position</code> , <code>pose.pose.orientation</code>	Robot location
<code>geometry_msgs/Twist</code>	<code>linear.x</code> , <code>angular.z</code>	Velocity commands

Because message types are standardised, any teleop node can drive any robot driver — as long as they agree on the topic name and message type.

Our teleop publishes `geometry_msgs/Twist` → RosAria consumes it → motors move.
Neither cares how the other is implemented.

The Data Collection Stack — Overview



Package: RosAria (Robot Driver)

What it is: A ROS node that wraps the ARIA library — the native SDK for Pioneer/P3DX robots.

What it does:

- Opens `/dev/ttyUSB0` (serial port to the robot's motor controller)
- Reads wheel encoder data → computes odometry → publishes pose
- Listens for velocity commands → sends them to the motors

Publishes:

```
/RosAria/pose
  nav_msgs/Odometry
  → x, y, z position
  → quaternion orientation
```

Subscribes:

```
/RosAria/cmd_vel
  geometry_msgs/Twist
  → linear.x (m/s forward)
  → angular.z (rad/s turn)
```

Built from source inside Docker because it is not in the default Noetic apt repository.

Package: realsense2_camera (Camera Driver)

What it is: Intel's official ROS wrapper around `librealsense2`.

What it does:

- Opens the RealSense D455 over USB 3.x
- Grabs colour (and optionally depth) frames
- Publishes them as standard ROS image messages

Publishes:

```
/camera/color/image_raw    ← RGB frames (sensor_msgs/Image)
/camera/aligned_depth_to_color/image_raw ← depth (optional)
```

Started with `roslaunch` (not `roslaunch`) because the launch file configures resolution, frame rate, and alignment in one go:

```
roslaunch realsense2_camera rs_camera.launch \
  color_width:=320 color_height:=240 color_fps:=15 align_depth:=true
```

Requires USB 3.x. A USB-C cable rated for USB 2.0 only will detect the camera but streaming will fail.

Scripts: Teleop Nodes

Two teleop scripts, same job — read input, publish velocity.

teleop_keyboard.py

- Puts the terminal in raw mode (`tty.setcbreak`)
- Reads single keypresses without Enter
- Maps `w/s/a/d/space` → Twist values
- Publishes at 20 Hz

teleop_joystick.py

- Reads Linux joystick events from `/dev/input/js0`
- Maps axes → linear/angular with deadzone + centre calibration
- Can run keyboard input in the same loop (`--keyboard` flag)
- Publishes at 20 Hz

Both publish `geometry_msgs/Twist` to `/RosAria/cmd_vel`.

RosAria picks it up and moves the motors. The scripts know nothing about the hardware.

```
twist = Twist()
twist.linear.x = forward_speed    # m/s
twist.angular.z = turn_rate       # rad/s
self.pub.publish(twist)
```

Script: record_data.py

The recorder subscribes to all three topics simultaneously and saves synchronised data at a fixed rate (default 1.5 Hz).

```
rospy.Subscriber("/camera/color/image_raw", Image, self.rgb_callback)
rospy.Subscriber("/RosAria/pose", Odometry, self.odom_callback)
rospy.Subscriber("/RosAria/cmd_vel", Twist, self.cmd_callback)
```

Each callback just stores the latest value in memory (protected by a `Lock` because ROS callbacks run on a background thread).

The main loop wakes up at `poll_hz` (30 Hz), checks if enough time has passed since the last save, then writes:

```
images/000042.jpg      ← RGB frame resized to 320×240
frames.jsonl           ← {"frame_idx":42, "stamp":..., "odom":{...}, "cmd":{...}}
dataset_meta.json      ← recording parameters (written on shutdown)
```

`cv_bridge` converts `sensor_msgs/Image` → numpy array so OpenCV can encode it as JPEG.

How a Single Frame Gets Recorded

```
RealSense camera
  | USB 3.x
  ▼
realsense2_camera node
  | publishes sensor_msgs/Image to /camera/color/image_raw
  ▼
record_data.py rgb_callback()
  | stores latest_rgb + timestamp in memory
  |
  | (meanwhile, at 1.5 Hz)
  ▼
record_data.py save_latest()
  |— cv_bridge converts Image → numpy array
  |— cv2.resize(rgb, (320, 240))
  |— cv2.imwrite("images/000042.jpg", bgr, quality=95)
  |— writes JSON line: {frame_idx, stamp, odom, cmd}
                        ▲           ▲
                        |           |
/RosAria/pose ————|
/RosAria/cmd_vel ————|
```

Docker — Why and How

Why

- ROS Noetic, RosAria, and the RealSense driver have many system-level dependencies
- Installing them directly on the robot laptop risks breaking other system packages
- The Docker image freezes a known-working environment

Key run flags

```
--network host
```

ROS uses the host's IP for node discovery. Docker's default NAT breaks this.

```
--privileged -v /dev:/dev
```

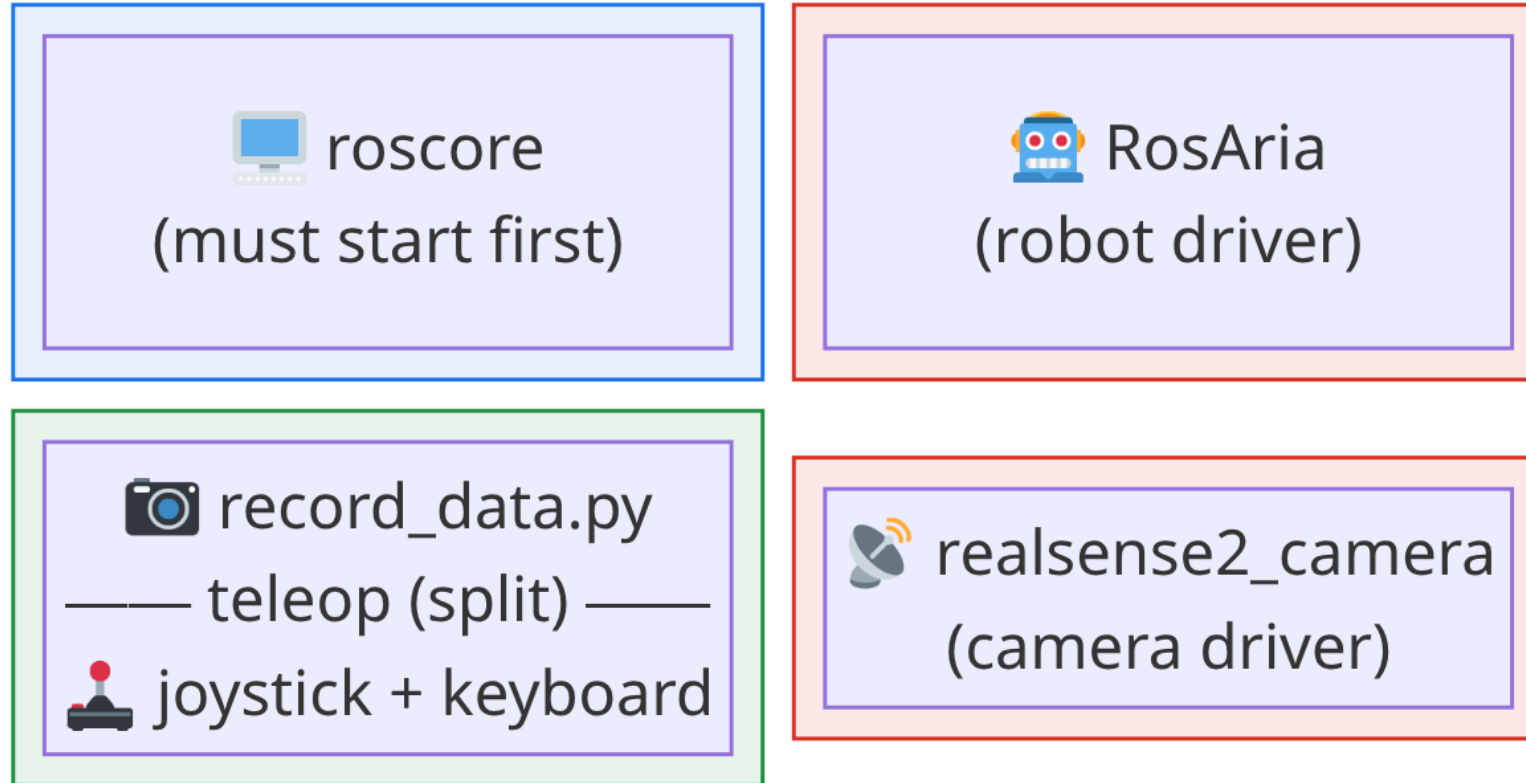
Gives the container access to USB devices (`/dev/ttyUSB0` , `/dev/input/js0` , RealSense).

```
-v "$PWD:/workspace/..."
```

The repo is mounted live — no rebuild needed for script changes.

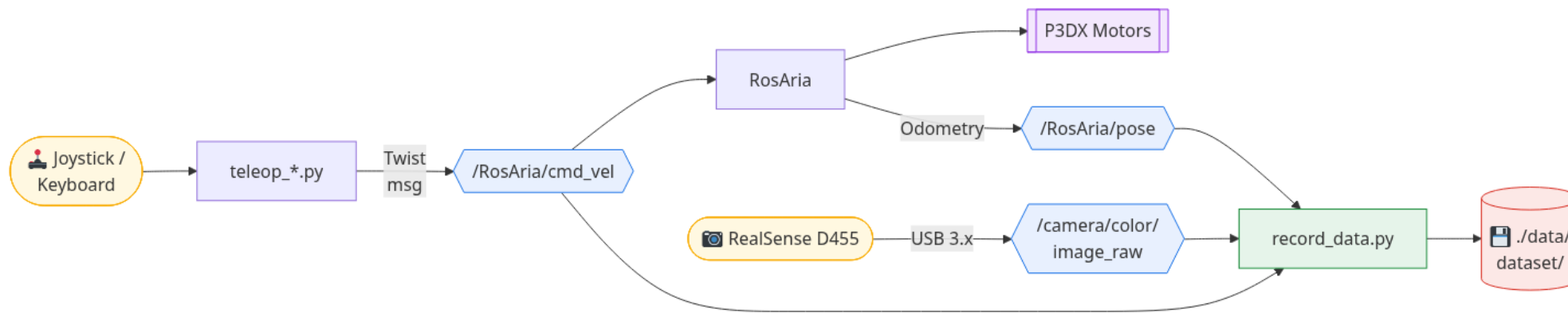
tmux Layout

`start_tmux.sh` creates one tmux window with four panes, each `docker exec`-ing into the container. All four processes run inside the same container, talking to each other through ROS topics via `roscore`.



Putting It All Together

Every 1/1.5 seconds, a snapshot of **what the robot saw**, **where it was**, and **what command was sent** is written to disk — a complete record for training or analysis.



Summary

Component	Role
<code>roscore</code>	Name registry — must run first
<code>RosAria</code>	Bridges serial port ↔ ROS topics
<code>realsense2_camera</code>	Bridges USB camera ↔ ROS image topic
<code>teleop_*.py</code>	Reads input → publishes <code>/RosAria/cmd_vel</code>
<code>record_data.py</code>	Subscribes to all topics → writes dataset
Docker	Isolates and packages the entire ROS environment
tmux	Runs all nodes in one visible terminal window